

S&DS 265 / 565

Introductory Machine Learning

Language Models

February 23

Yale

Business items

- Assignment 3 is out
- Quiz 3 on Wednesday

Midterm review sessions

Ruiqi: Friday, Feb 27, 7:00pm

Jianliang: Saturday, Feb 28, 5:00pm

Dingyao: Monday, March 2, 8:00pm

John: Tuesday, March 3, 5:30pm

Locations TBA

Invitation: Panel discussion

Class on Monday, April 20:

We will have a panel discussion on

Societal issues for AI and Machine Learning

If you would like to volunteer to participate, please email us:

To: `sds265@yale.edu`

Subject: `iML panel`

For Today

- Language models
- Concepts...no new methods

Language models

- A language model is a way of *generating* any sequence of words

$$\begin{aligned} P(\text{"the whole forest had been anesthetized"}) = & \\ & P(\text{"the"}) \times P(\text{"whole"} \mid \text{"the"}) \\ & \times P(\text{"forest"} \mid \text{"the whole"}) \\ & \times P(\text{"had"} \mid \text{"the whole forest"}) \\ & \times P(\text{"been"} \mid \text{"the whole forest had"}) \\ & \times P(\text{"anesthetized"} \mid \text{"the whole forest had been"}) \end{aligned}$$

Remixing Noon

Text generated from Channel Skin by Jeff Noon

"The whole forest had been anesthetised, her temples wired, her senses stimulated, her eyes were not on Eva, not on Eva, not on Eva, not on anybody in that same realm, the land of dreams and nightmares."

Viability: 0.000000326%



<https://chatbotslife.com/notes-on-remixing-noon-generative-text-and-markov-chains-84ff4ec23937>

Text generation

- Words generated one-by-one
- A word is chosen by sampling from a probability distribution
- Result is purely synthetic text—generative AI

Uses of language models

- Speech recognition
- Machine translation
- Text compression
- Texting
- Email completion
- Image captioning
- Mind reading from fMRI

Dasher: LMs for assistive devices

Language models enable new modes of text input:

https://www.youtube.com/watch?v=quw_Kci4fUg

Language models

- A language model is a way of assigning a probability to any sequence of words (or string of text)

$$p(w_1, \dots, w_n)$$

Language models

- A language model is a way of assigning a probability to any sequence of words (or string of text)

$$p(w_1, \dots, w_n)$$

- By the basic rules of conditional probability we can factor this as

$$p(w_1, \dots, w_n) = p(w_1)p(w_2 | w_1) \dots p(w_n | w_1, \dots, w_{n-1})$$

Language models

- A language model is a way of assigning a probability to any sequence of words (or string of text)

$$p(w_1, \dots, w_n)$$

- By the basic rules of conditional probability we can factor this as

$$p(w_1, \dots, w_n) = p(w_1)p(w_2 | w_1) \dots p(w_n | w_1, \dots, w_{n-1})$$

- The number of *histories* grows as V^{n-1} . Number of parameters in model grows as V^n , where V is number of words in vocabulary.

Language models

- A language model is a way of assigning a probability to any sequence of words (or string of text)

$$p(w_1, \dots, w_n)$$

- By the basic rules of conditional probability we can factor this as

$$p(w_1, \dots, w_n) = p(w_1)p(w_2 | w_1) \dots p(w_n | w_1, \dots, w_{n-1})$$

- The number of *histories* grows as V^{n-1} . Number of parameters in model grows as V^n , where V is number of words in vocabulary.
- What are some ways of reducing the number of parameters?

One approach: Grouping histories

- Let $g(w_1, \dots, w_n)$ be the group assigned to the history

One approach: Grouping histories

- Let $g(w_1, \dots, w_n)$ be the group assigned to the history
- Our model becomes

$$p(w_{n+1} \mid w_1, \dots, w_n) = p(w_{n+1} \mid g(w_1, \dots, w_n))$$

One approach: Grouping histories

- Let $g(w_1, \dots, w_n)$ be the group assigned to the history
- Our model becomes

$$p(w_{n+1} \mid w_1, \dots, w_n) = p(w_{n+1} \mid g(w_1, \dots, w_n))$$

- Number of parameters: $O(V \cdot \text{number of groups})$

One approach: Grouping histories

- Let $g(w_1, \dots, w_n)$ be the group assigned to the history
- Our model becomes

$$p(w_{n+1} \mid w_1, \dots, w_n) = p(w_{n+1} \mid g(w_1, \dots, w_n))$$

- Number of parameters: $O(V \cdot \text{number of groups})$
- What are some example groupings?

Grouping histories

- Unigrams: $g(w_1, \dots, w_n) = \emptyset$.

The notation $O(\cdot)$ is called “Big Oh” and means “no greater than a constant times”, so that $O(f(n))$ means a sequence that is bounded by $C \cdot f(n)$ for large enough n .

Grouping histories

- Unigrams: $g(w_1, \dots, w_n) = \emptyset$.
- Bigrams: $g(w_1, \dots, w_n) = w_n$.

The notation $O(\cdot)$ is called “Big Oh” and means “no greater than a constant times”, so that $O(f(n))$ means a sequence that is bounded by $C \cdot f(n)$ for large enough n .

Grouping histories

- Unigrams: $g(w_1, \dots, w_n) = \emptyset$.
- Bigrams: $g(w_1, \dots, w_n) = w_n$.
- Trigrams: $g(w_1, \dots, w_n) = (w_{n-1}, w_n)$.

The notation $O(\cdot)$ is called “Big Oh” and means “no greater than a constant times”, so that $O(f(n))$ means a sequence that is bounded by $C \cdot f(n)$ for large enough n .

Grouping histories

- Unigrams: $g(w_1, \dots, w_n) = \emptyset$.
- Bigrams: $g(w_1, \dots, w_n) = w_n$.
- Trigrams: $g(w_1, \dots, w_n) = (w_{n-1}, w_n)$.
- Number of parameters grows as $O(V)$, $O(V^2)$, and $O(V^3)$, respectively.

The notation $O(\cdot)$ is called “Big Oh” and means “no greater than a constant times”, so that $O(f(n))$ means a sequence that is bounded by $C \cdot f(n)$ for large enough n .

Estimating parameters

- The maximum likelihood estimate of a trigram model:

$$\hat{p}(w_3 | w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$$

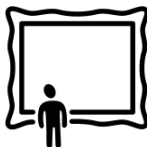
Estimating parameters

- The maximum likelihood estimate of a trigram model:

$$\hat{p}(w_3 | w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$$

- What are some problems with this model?

Sparse data problem



The next group of slides presents one way of quantifying the problem of sparse data in language modeling

Half Earth

Aug 06, 2018 by Foundation Staff 0 Comment Google Earth, Half-Earth Project, Map of Life

By Jeremy Malczyk, Michelle Duong, Ajay Ranipeta, Chris Heltne, Walter Jetz of Map of Life, Yale University, and the E.O. Wilson Biodiversity Foundation Half-Earth Project

This article originally in *Medium*, July 30, 2018



Learn how you can be part of bringing Half-Earth to life.

The Significance of Biodiversity



Narrow-billed Tody, *Todus angustirostris*. Photo by Julie Hart.

<https://eowilsonfoundation.org/mapping-species-for-half-earth/>

<https://www.half-earthproject.org/>

Specious species



- A naturalist (say, Edward O. Wilson) explores a region and observes animals (organisms).

Specious species



- A naturalist (say, Edward O. Wilson) explores a region and observes animals (organisms).
- He finds 100,000 animals and 5,000 species.

Specious species



- A naturalist (say, Edward O. Wilson) explores a region and observes animals (organisms).
- He finds 100,000 animals and 5,000 species.
- What is the chance I'll find a new species?

Specious species



- A naturalist (say, Edward O. Wilson) explores a region and observes animals (organisms).
- He finds 100,000 animals and 5,000 species.
- What is the chance I'll find a new species?
- What if Wilson observes 100 unique species?

Missing species: Good-Turing

- Wilson observes 100,000 animals and 5,000 species, 100 of them are unique (only one observation of that species).

Missing species: Good-Turing

- Wilson observes 100,000 animals and 5,000 species, 100 of them are unique (only one observation of that species).
- Good-Turing: Estimate of the probability that the next animal is a new species? $\hat{p}_{GT} = 100/100,000 = 10^{-3}$.

Missing species: Good-Turing

- Wilson observes 100,000 animals and 5,000 species, 100 of them are unique (only one observation of that species).
- Good-Turing: Estimate of the probability that the next animal is a new species? $\hat{p}_{GT} = 100/100,000 = 10^{-3}$.
- This is an estimate of the missing probability mass.

Sparse data: Species $\approx$ words

- Suppose we have a corpus of 500 million words. I count trigrams and find that 50 million of them are unique.

Sparse data: Species $\approx$ words

- Suppose we have a corpus of 500 million words. I count trigrams and find that 50 million of them are unique.
- When I see a new trigram, 10% of the time it won't have been seen before. (This is a typical number.)

Sparse data: Species $\approx$ words

- Suppose we have a corpus of 500 million words. I count trigrams and find that 50 million of them are unique.
- When I see a new trigram, 10% of the time it won't have been seen before. (This is a typical number.)
- This means that the maximum likelihood estimate (MLE) is zero, and the probability that my model predicts the next word will be zero.

Sparse data: Species $\approx$ words

- Suppose we have a corpus of 500 million words. I count trigrams and find that 50 million of them are unique.
- When I see a new trigram, 10% of the time it won't have been seen before. (This is a typical number.)
- This means that the maximum likelihood estimate (MLE) is zero, and the probability that my model predicts the next word will be zero.
- The MLE is supported on the observed data. We need to spread out the probability over unseen events.

Estimating parameters

- The maximum likelihood estimate of a trigram model:

$$\hat{p}(w_3 | w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$$

Estimating parameters

- The maximum likelihood estimate of a trigram model:

$$\hat{p}(w_3 | w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$$

- How can the model be strengthened?

Smoothing and Interpolation

Smoothing:

$$\tilde{p}(w_3 | w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3) + \varepsilon}{\text{count}(w_1, w_2) + V\varepsilon}$$

Smoothing and Interpolation

Linear interpolation:

$$p(w_3 | w_1, w_2) = \lambda_3 \hat{p}(w_3 | w_1, w_2) + \lambda_2 \hat{p}(w_3 | w_2) + \lambda_1 \hat{p}(w_3)$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$.

This is a type of “mixture model”

Modern language models

The algorithm assigns a “score” to possible next words v :

$$\begin{array}{c} \text{previous words} \\ s(v; \overbrace{w_1, \dots, w_n}) \\ \uparrow \\ \text{possible next word} \end{array}$$

The score is given by

$$s(v; w_1, \dots, w_n) = \beta_v^T \varphi(w_1, \dots, w_n)$$

where $\varphi(w_1, \dots, w_n)$ is an encoding of the context into a big vector, and β_v is an “embedding” of word v .

Converted into a probability with softmax

Distributed representations

So, histories are mapped to *embeddings*

How good is a language model?

The next group of slides presents a useful way of quantifying how good a language model is

Recall: Geometric mean

The *arithmetic mean* of $1/4$, 4 , 8 is

$$\frac{1}{3} \left(\frac{1}{4} + 4 + 8 \right) = 4.0\overline{83}$$

Recall: Geometric mean

The *arithmetic mean* of $1/4$, 4 , 8 is

$$\frac{1}{3} \left(\frac{1}{4} + 4 + 8 \right) = 4.0\overline{83}$$

The *geometric mean* of $1/4$, 4 , 8 is

$$\sqrt[3]{\frac{1}{4} \cdot 4 \cdot 8} = 2$$

Recall: Geometric mean

The *arithmetic mean* of $1/4$, 4 , 8 is

$$\frac{1}{3} \left(\frac{1}{4} + 4 + 8 \right) = 4.0\overline{83}$$

The *geometric mean* of $1/4$, 4 , 8 is

$$\sqrt[3]{\frac{1}{4} \cdot 4 \cdot 8} = 2$$

The geometric mean is no greater than the arithmetic mean

Recall: Geometric mean

The *geometric mean* of $x_1, \dots, x_n$ is

$$\sqrt[n]{x_1 x_2 \cdots x_n} = \left(\prod_{i=1}^n x_i \right)^{\frac{1}{n}}$$

Recall: Geometric mean

The *geometric mean* of $x_1, \dots, x_n$ is

$$\sqrt[n]{x_1 x_2 \cdots x_n} = \left(\prod_{i=1}^n x_i \right)^{\frac{1}{n}}$$

How good is a language model? Perplexity

Perplexity is defined as

$$\text{Perplexity}(\theta) = \left(\prod_{i=1}^N p_{\theta}(w_i \mid w_{1:i-1}) \right)^{-\frac{1}{N}}$$

where $w_1, w_2, \dots, w_N$ is a large chunk of text that wasn't used to train the language model.

How good is a language model? Perplexity

- Perplexity is the inverse of the geometric mean of the word probabilities
- Intuitively, if the perplexity is 100, the actual next word is in the top 100 words of the model (on average).
- This is the (geometric) average “branching factor” for the model on real text

perplexity

Ask anything...



Model ▾



💰 Finance

✈️ Travel

🔧 Troubleshoot

♥️ Health

🔍 Research

Recall: Embeddings

We discussed embeddings as constructed from simple bigram language models:

$$\log \mathbb{P}_{\text{near}}(w_1, w_2) = \varphi(w_1)^T \varphi(w_2) + \text{constant}$$

where $\varphi(w)$ is the *embedding vector* for word w .

Learning embeddings

- This language model is trained using stochastic gradient descent.
- If we have $d = 500$ dimensional embeddings, there are 500 trainable parameters for *each* word
- How are these numbers adjusted (trained)?

Learning embeddings: SGD

Start with *random* embeddings. Then iterate many, many times:

- 1 Sample a pair of words w_1 and w_2 that appear nearby
- 2 Adjust the embedding parameters of each word:

$$\varphi(w_1) = \varphi(w_1) + \eta \varphi(w_2)$$

$$\varphi(w_2) = \varphi(w_2) + \eta \varphi(w_1)$$

The vectors are brought closer together in each step!

Learning embeddings: SGD

Why is this? The *loss* is the *negative* log-probability:

$$-\log \mathbb{P}_{\text{near}}(\mathbf{w}_1, \mathbf{w}_2) = -\varphi(\mathbf{w}_1)^T \varphi(\mathbf{w}_2) + \text{constant}$$

The derivative of this loss with respect to $\varphi(\mathbf{w}_1)$ is $-\varphi(\mathbf{w}_2)$

SGD update adds a small step size η times *minus* the derivative:

$$\varphi(\mathbf{w}_1) = \varphi(\mathbf{w}_1) + \eta \varphi(\mathbf{w}_2)$$

Modern language models

An LLM assigns a “score” to possible next words:

$$s(v; \underbrace{w_1, \dots, w_n}_{\text{word history}})$$

Modern language models

An LLM assigns a “score” to possible next words:

$$s(v; \underbrace{w_1, \dots, w_n}_{\text{word history}})$$

Converted to a probability by the “softmax” operation:

$$p(w \mid w_1, \dots, w_n) = \frac{\exp(s(w; w_1, \dots, w_n))}{\sum_{v \in V} \exp(s(v; w_1, \dots, w_n))}$$

Modern language models

An LLM assigns a “score” to possible next words:

$$s(v; \underbrace{w_1, \dots, w_n}_{\text{word history}})$$

Converted to a probability by the “softmax” operation:

$$p(w \mid w_1, \dots, w_n) = \frac{\exp(s(w; w_1, \dots, w_n))}{\sum_{v \in V} \exp(s(v; w_1, \dots, w_n))}$$

The function $s(v; w_{1:n})$ is learned on large amounts of text (unsupervised) using a type of deep neural network called a *transformer*.

Modern language models

In fact, the score is computed as

$$s(v; \underbrace{w_1, \dots, w_n}_{\text{word history}}) = \varphi(v)^T g(w_1, \dots, w_n)$$

- the “grouping function” $g(w_1, \dots, w_n)$ is like an embedding, that maps the word history to a high dimensional vector
- The vector $\varphi(v)$ is an embedding of word v

We'll see how this is done when we discuss neural networks

Summary of today: Language models

- A language model is used to predict or generate the next word
- Used in many different applications
- Probabilities need to be “smoothed” to avoid zeros
- Perplexity is a measure of a language model’s predictive power